TEMCO LEARN

T3000 API & Protocol

T3 Firmware — LVGL Display Architecture · built Nov 13, 2026

T3 Firmware — LVGL Display Architecture

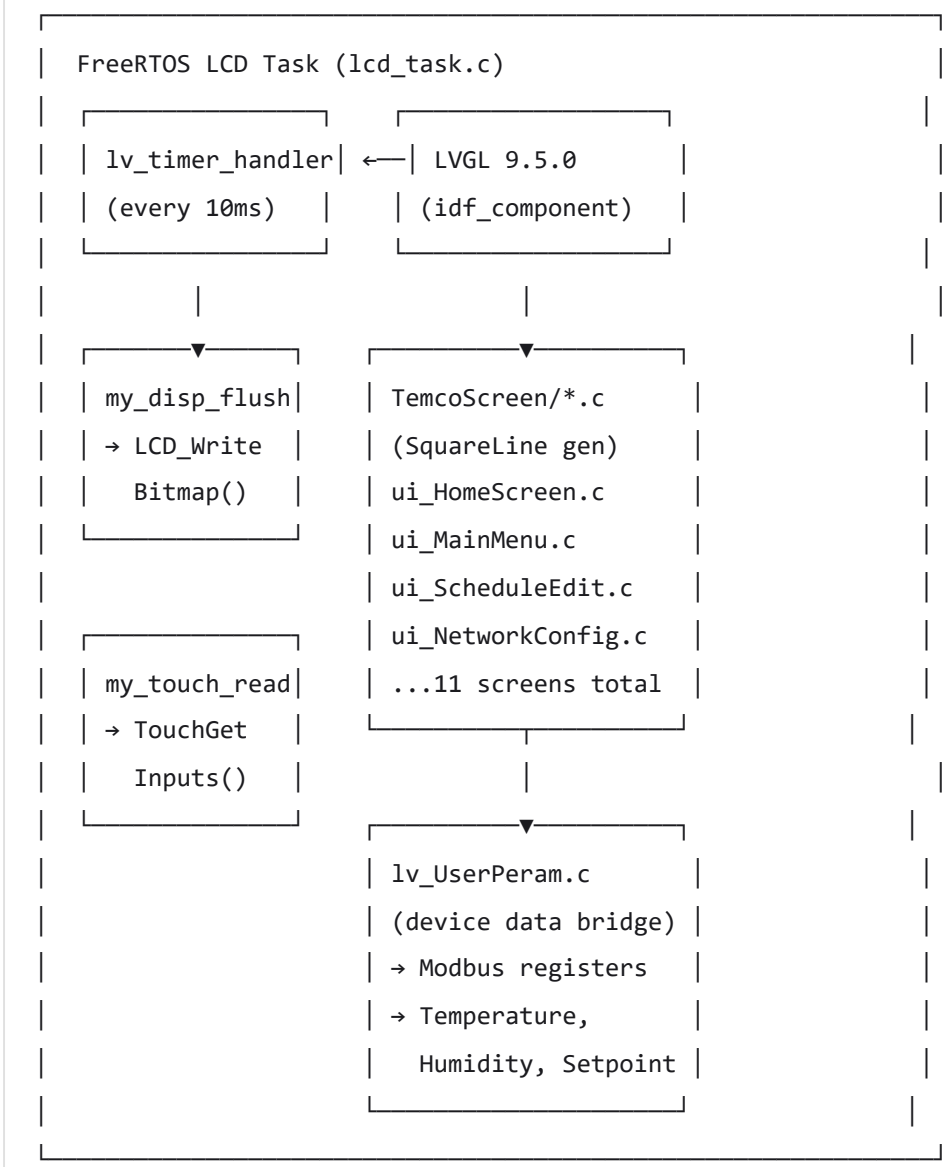
Part 1 — Current Firmware (SquareLine Studio)

Branch: `Bhavik-Tstate-11` **Tool:** SquareLine Studio 1.6.0 **LVGL:** 9.5.0 (via ESP-IDF component manager)

Display: ILI9341 320×240 via hardware SPI + external touch

Architecture Overview

TEXT



---

1. How Screens Are Created (SquareLine Studio)

SquareLine Studio generates static C code. Each screen is a function that calls `lv_obj_create()` etc.:

```c
// ui_HomeScreen.c — generated by SquareLine Studio 1.6.0
void ui_HomeScreen_screen_init(void) {
    ui_HomeScreen = lv_obj_create(NULL);
    lv_obj_set_size(ui_HomeScreen, 320, 240);

    // Create temperature label
    ui_TemperatureVal = lv_label_create(ui_HomeScreen);
    lv_label_set_text(ui_TemperatureVal, "22.5");
    lv_obj_set_pos(ui_TemperatureVal, 80, 20);
    lv_obj_set_style_text_font(ui_TemperatureVal, &ui_font_Arial80, LV_PART_MAIN);

    // Create fan mode button
    ui_AutoBtn = lv_btn_create(ui_HomeScreen);
    lv_obj_set_pos(ui_AutoBtn, 200, 150);
    // ...add label, image, event handler
}
```

All screens are created at boot in `ui_init()`:

```c
void ui_init(void) {
    ui_StartUpScreen_screen_init();   // Boot/splash
    ui_HomeScreen_screen_init();      // Main thermostat
    ui_MainMenu_screen_init();        // Settings menu
    ui_WifiConfig_screen_init();      // WiFi settings
    ui_NetworkConfig_screen_init();   // Network/IP settings
    ui_Protocols_screen_init();       // BACnet/Modbus settings
    ui_Parameters_screen_init();      // Parameter viewer
    ui_Time_screen_init();            // Time/clock
    ui_ScheduleScreen_screen_init();  // Weekly schedule
    ui_ScheduleEditScreen_screen_init(); // Schedule editor
    ui_HolidayCalenderScreen_screen_init(); // Holidays
    lv_scr_load(ui_StartUpScreen);    // Show boot screen first
}
```

---

2. Screen Navigation

Screen switching is hardcoded, typically from event callbacks in `ui_events.c`:

```c
// ui_events.c — event handler that switches screens
void FanSetAutoMode(lv_event_t * e) {
    Event_Cb_FanSetAutoMode(e);    // device logic
    _ui_screen_change(&ui_MainMenu, LV_SCR_LOAD_ANIM_FADE_ON, 200, 0, &ui_MainMenu_scree
}
```

Or directly via `lv_scr_load()`:

```c
lv_scr_load(ui_HomeScreen);
```

No navigation graph or flow engine — each screen knows its own navigation targets.

---

3. Device Data Binding (lv_UserPeram.c)

The bridge between LVGL UI and Modbus device data:

```c
// lv_UserPeram.c
void UpdateTemperatureDisplay(void) {
    // Read from Modbus register
    int32_t temp = Temperature_InVal / 10;
    char buf[16];
    sprintf(buf, "%d.%d°C", temp, abs(Temperature_InVal % 10));
    lv_label_set_text(ui_TemperatureVal, buf);
}

// Polled periodically in the LCD task's timer callback
void lv_UserPeram_Poll(void) {
    if (Temperature_InVal != last_temp) {
        UpdateTemperatureDisplay();
        last_temp = Temperature_InVal;
    }
    if (Humidity_InVal != last_hum) {
        UpdateHumidityDisplay();
        last_hum = Humidity_InVal;
    }
    // ... fan mode, system mode, time, WiFi status, etc.
}
```

Data flow: `Modbus RTU → deviceparams → lv_UserPeram → LVGL objects`

---

4. Display Driver (lcd_task.c + lcd_drv.c)

- **SPI**: Hardware SPI via ESP-IDF `spi_master` driver (pins: MOSI, MISO, CLK, CS, DC)
- **Buffer**: Full-frame buffer in PSRAM (`MALLOC_CAP_SPIRAM`)
- **Flush**: `my_disp_flush()` → `LCD_WriteBitmap()` → SPI DMA transfer
- **Touch**: I2C touch controller via `TouchGetInputs()` → LVGL input device
- **Tick**: `esp_timer_get_time() / 1000` for `lv_tick_set_cb()`
- **Task**: FreeRTOS task at 10ms tick — calls `lv_timer_handler()` + `lv_UserPeram_Poll()`

C

```c
void lcd_task(void *arg) {
    LCD_Init();                // ILI9341 init + backlight
    TouchInit();               // I2C touch init
    LCD_RegesterlvglDriver();  // lv_display_create + flush/read callbacks
    ui_init();                 // Create all SquareLine screens

    while (1) {
        lv_timer_handler();        // LVGL rendering
        lv_UserPeram_Poll();       // Update device data
        T3_Lv_Scr_Manager();       // Screen auto-rotate/timer
        vTaskDelay(pdMS_TO_TICKS(LCD_TASK_DELAY_MS));  // 10ms
    }
}
```

---

5. Build System

YAML

```yaml
## main/idf_component.yml
dependencies:
  idf:
    version: '>=5.5.3'
  espressif/led_strip: '*'
  lvgl/lvgl:
    version: "==9.5.0"      # Pulled by ESP-IDF component manager
```

CMAKE

```
## main/CMakeLists.txt
idf_component_register(SRCS
    "LCD_Driver/lcd_drv.c"
    "LCD_Driver/lcd_task.c"
    "LCD_Driver/touch_drv.c"
    "LCD_Driver/lv_UserPeram.c"
    ...
)
## Auto-include all SquareLine generated .c files
file(GLOB TEMCO_SCREEN_SOURCES "TemcoScreen/*.c")
target_sources(${COMPONENT_LIB} PRIVATE ${TEMCO_SCREEN_SOURCES})
```

---

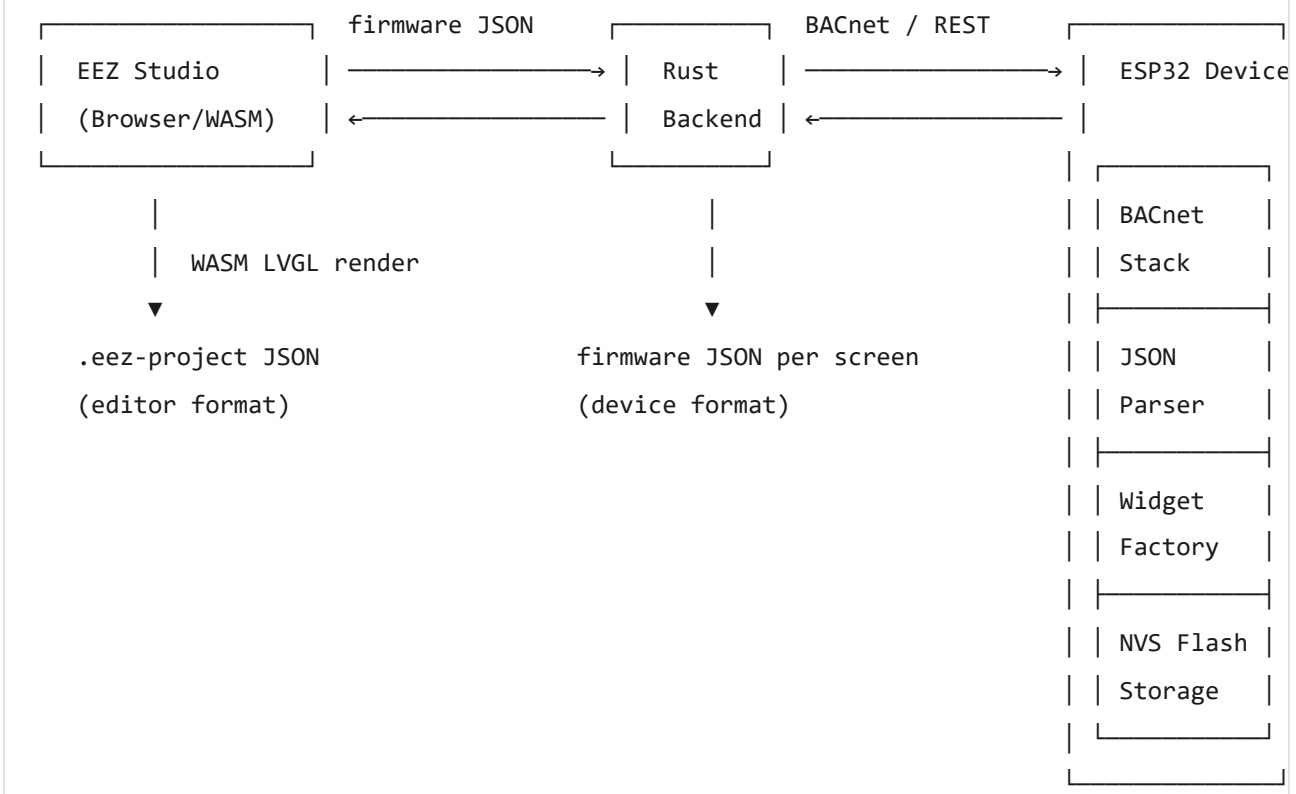
6. Screens List

File	Screen	Lines	Purpose
`ui_StartUpScreen.c`	Boot splash	~50	Logo + initialization
`ui_HomeScreen.c`	Main thermostat	868	Temperature, humidity, fan, mode
`ui_MainMenu.c`	Settings menu	349	Navigation to sub-screens
`ui_WifiConfig.c`	WiFi settings	326	SSID, password, scan
`ui_NetworkConfig.c`	Network settings	769	IP, subnet, BACnet port
`ui_Protocols.c`	Protocol config	371	BACnet, Modbus settings
`ui_Parameters.c`	Parameter viewer	151	Device parameters
`ui_Time.c`	Clock settings	557	Date/time/schedule
`ui_ScheduleScreen.c`	Schedule overview	666	Weekly program list
`ui_ScheduleEditScreen.c`	Schedule editor	2,608	Day/time/temp editing
`ui_HolidayCalenderScreen.c`	Holiday calendar	107	Holiday date picker

Part 2 — EEZ Studio Project Flow (Web ↔ Device)

Architecture

TEXT



2.1 Two JSON Formats

Format	Producer	Consumer	Content
`.eez-project`	EEZ Studio save	Editor, build system	Full project: settings, pages, styles, fonts, bitmaps (base64), actions, connectionLines
Firmware JSON	`firmware-export.ts`	Device firmware, mock store	Per-screen: widgets flattened, actions resolved, bitmaps as names only

2.2 Export: .eez-project → Firmware JSON

`firmware-export.ts` transforms the editor format to per-screen device JSON:

```
TEXT
```

```
.eez-project                          Firmware JSON (per screen)
____________                          ____________________________

userPages[]                             screens[]
  └─ components[]                        └─ widgets[]
       ├─ LVGLScreenWidget (unwrapped)          ├─ { sub_type: "label", ... }
       ├─ LVGLLabelWidget                       ├─ { sub_type: "button", ... }
       ├─ LVGLButtonWidget                      └─ { sub_type: "action", ... }
       ├─ LVGLActionComponent
       └─ connectionLines (resolved)       events attached directly:
                                               CLICKED → actions: [changeScreen(...)]
```

Field mapping (export):

`.eez-project` field	Firmware JSON field
`left` / `top`	`x_pos` / `y_pos`
`width` / `height`	`width` / `height`
`text` / `textType`	`obj_text` / `text_type`
`localStyles.definition`	`style`
`eventHandlers[]`	`events: { TRIGGER: { actions: [...] } }`
`actions[].action: "changeScreen"`	`actions: [{ action: "changeScreen", screen: "..." }]`
`children[]`	`children: { ... }` (recursive)

Action resolution: `connectionLines` linking widget events → `LVGLActionComponent` are resolved at export time. Actions are attached directly to widget events in the firmware JSON. Standalone action components are stripped.

2.3 Transfer: Browser ↔ Device

Two paths (auto-detected by `DeviceRestClient`):

Path	Direction	Endpoint	Transport
REST (primary)	Push	`PUT /api/v1/screens`	HTTP to ESP32 port 8000
	Pull	`GET /api/v1/screens`	HTTP from ESP32 port 8000
BACnet (fallback)	Push	`POST /api/eez-device/screens/push/:panelId`	HTTP → Rust → BACnet → Device
	Pull	`POST /api/eez-device/screens/pull/:panelId`	HTTP → Rust → BACnet → Device

API surface (`/api/eez-device/*`):

Method	Endpoint	Purpose
`GET`	`/screens`	Load all screens
`PUT`	`/screens`	Deploy all screens
`GET`	`/screens/:name`	Load single screen
`PUT`	`/screens/:name`	Deploy single screen
`PATCH`	`/screens/:name`	Delta update
`PATCH`	`/screens/:name/widgets/:id`	Widget delta
`POST`	`/screens/push/:panelId`	BACnet push
`POST`	`/screens/pull/:panelId`	BACnet pull
`POST`	`/images/push/:panelId`	Upload bitmap
`GET`	`/images/pull/:panelId/:name`	Download bitmap
`DELETE`	`/images/:panelId/:name`	Remove bitmap
`GET`	`/devices`	List devices

2.4 Import: Firmware JSON → .eez-project

`firmware-loader.ts` reverses the export transform:

```
TEXT
```

```
Firmware JSON (per screen)          .eez-project
──────────────────────────          ────────────

widgets[]                             userPages[]
  ├─ { sub_type: "label", ... }        └─ components[]
  ├─ { sub_type: "button", ... }            ├─ LVGLLabelWidget
  └─ events: { CLICKED: { actions } }       ├─ LVGLButtonWidget
       ↓                                     ├─ LVGLActionComponent (rebuilt)
  LVGLActionComponent created                └─ connectionLines (rebuilt)
  connectionLine created
```

Field mapping (import):

Firmware JSON field	`.eez-project` field
`x_pos` / `y_pos`	`left` / `top`
`obj_text` / `text_type`	`text` / `textType`
`style`	`localStyles.definition`
`events[].actions[]`	`LVGLActionComponent` + `connectionLine`
`children`	`children` / `components` (recursive)
`min` / `max` / `value`	`min` / `max` / `value` / `valueType`
`src` (image)	`image`
`checked` / `checked_type`	`checkedState` / `checkedStateType`
`long_mode` / `recolor`	`longMode` / `recolor`
`inner_align`	`innerAlign`
`options` / `selected`	`options` / `selected`
`widget` (user_widget)	`userWidgetPageName`

2.5 Frontend Rendering (Browser)

LVGL WASM runtime renders `.eez-project` pages in the browser:

```
.eez-project JSON
      ↓
ProjectStore.create() → loadProject() → setProject()
      ↓
LVGL WASM runtime (Emscripten-compiled)
  ├─ v8.4.0, v9.2.2, v9.3.0, v9.4.0, v9.5.0
  └─ Served from /eez-studio-wasm/wasm/lvgl/{version}/
      ↓
Canvas-based LVGL display buffer → browser-rendered preview
```

Runtime loading (from `lvgl-versions.ts`):

1. Detects browser environment
2. Loads JS glue via `<script>` tag (sets `globalThis.LVGLWasmRuntime`)
3. Instantiates Emscripten module with LVGL compiled to WASM
4. Creates virtual display in an HTML canvas

Supported LVGL versions: v8.4.0, v9.2.2, v9.3.0, v9.4.0, v9.5.0 (default for new projects).

2.6 Full Round-Trip Flow

```
DEPLOY:                                IMPORT:
EEZ Studio                             Open Page
  │ "Deploy to Device"                 │ "Import from Device"
  ▼                                    ▼
firmware-export.ts                     DeviceRestClient.connect()
  │ .eez-project → DeviceScreen[]      │ isReachable() → REST or BACnet
  ▼                                    ▼
DeviceRestClient.deployAllScreens()    GET /api/eez-device/screens
  │ PUT /api/eez-device/screens        │
  ▼                                    ▼
BACnet → ESP32 NVS                     firmware-loader.ts
  │ device stores + parses             │ Firmware JSON → .eez-project
  ▼                                    ▼
Device creates LVGL widgets            WASM LVGL render in browser
```

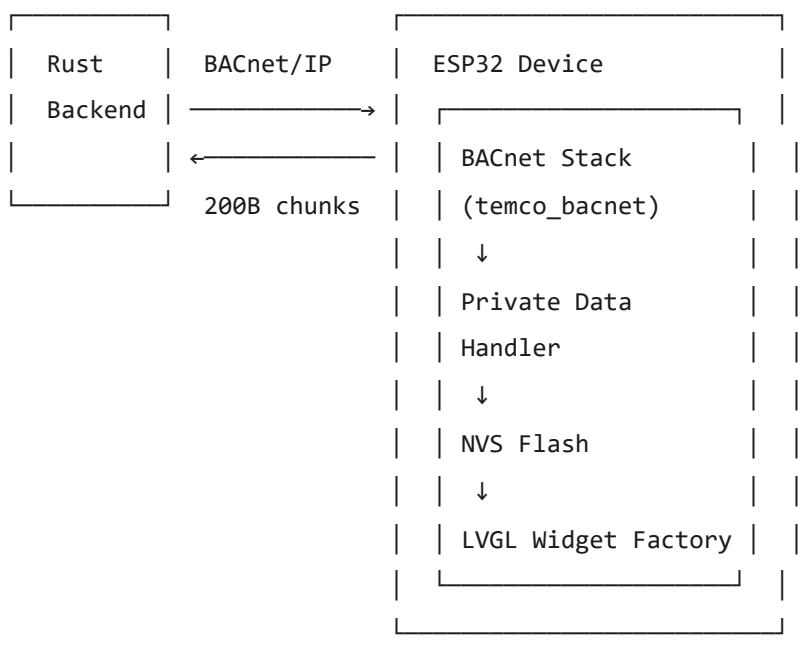
2.7 Device BACnet API

When the real device supports EEZ Studio screens, it exposes a BACnet-based API for screen push/pull.

2.7.1 BACnet Transfer Protocol

Screens are transferred in **200-byte compressed chunks** via BACnet Private Data objects on the ESP32:

TEXT



BACnet Private Data Commands — implemented on the ESP32:

Command	Direction	Purpose
`WRITE_JSON_SCREEN`	Rust → Device	Screen metadata (name, zip size)
`WRITE_JSON_ITEM`	Rust → Device	200-byte chunk of compressed JSON
`READ_JSON_SCREEN`	Rust → Device	Request screen metadata
`READ_JSON_ITEM`	Rust → Device	Request 200-byte chunk
`WRITE_SETTING_COMMAND`	Rust → Device	Set `webview_json_flash = 2`

Deploy sequence:

1. Rust sends `WRITE_SETTING_COMMAND` → device sets `webview_json_flash = 2`
2. Rust sends `WRITE_JSON_SCREEN` → device allocates NVS space, prepares for chunks
3. Rust sends `WRITE_JSON_ITEM` × N (200 bytes each, 2 per BACnet write) → device assembles in NVS buffer
4. Device verifies CRC, decompresses with zlib, parses JSON
5. Device creates/destroys LVGL widgets from JSON definitions

Read sequence:

1. Rust sends `READ_JSON_SCREEN` → device returns screen metadata (names, sizes)
2. Rust sends `READ_JSON_ITEM` × N → device returns 200-byte chunks from NVS
3. Rust assembles chunks, decompresses, returns JSON to browser

2.7.2 NVS Flash Storage

Each screen uses **10 slots** of 200 bytes in NVS flash:

```
TEXT

Slot layout:  screenIndex × 10 + chunkIndex  (0..9)
Per screen:   10 slots × 200 bytes = 2 KB max compressed
Per device:   up to N screens in NVS partition
```

The `webview_json_flash` flag (Modbus register offset 253) indicates the device mode:

- `0` = legacy hardcoded UI
- `2` = EEZ Studio JSON mode

2.7.3 REST API (ESP32)

As an alternative to BACnet, the ESP32 hosts a lightweight HTTP server:

Method	Endpoint	Purpose
`GET`	`/api/v1/screens`	Return all stored screens
`PUT`	`/api/v1/screens`	Store screen definitions
`GET`	`/api/v1/screens/:name`	Single screen
`PUT`	`/api/v1/screens/:name`	Update single screen
`PATCH`	`/api/v1/screens/:name`	Delta update

`DeviceRestClient` probes REST first (port 8000, 2s timeout), falls back to BACnet if unreachable.

2.7.4 Device Firmware Modules

To support EEZ Studio JSON on the ESP32:

1. **BACnet Private Data handlers** for `WRITE_JSON_SCREEN`, `WRITE_JSON_ITEM`, `READ_JSON_SCREEN`, `READ_JSON_ITEM`
2. **Zlib decompression** — screens arrive compressed in 200-byte chunks

3. **JSON parser** (cJSON) — parse firmware JSON to extract widget definitions
4. **NVS storage** — persist screens in flash, survive reboots
5. **Widget factory** — map `sub_type` → `lv_obj_create()` at runtime
6. **Expression engine** — resolve `variables.temp` to Modbus register values
7. **Action engine** — handle `changeScreen`, `setVariable` actions
8. **Image loader** — convert base64 bitmaps to LVGL image descriptors
9. **REST server** — lightweight HTTP on port 8000 for direct access

---

3.1 Building BACnet/REST API on the ESP32

The device needs two API surfaces to receive and serve screen definitions.

3.1.1 BACnet Private Data API

The ESP32's existing BACnet stack (`temco_bacnet`) is extended with Private Data object handlers:

Commands implemented on the device:

Command	Direction	Handler	Purpose
`WRITE_JSON_SCREEN`	Client → Device	`bacnet_json_screen_write()`	Receive screen metadata (name, compressed size)
`WRITE_JSON_ITEM`	Client → Device	`bacnet_json_item_write()`	Receive 200-byte compressed chunk
`READ_JSON_SCREEN`	Client → Device	`bacnet_json_screen_read()`	Return screen metadata
`READ_JSON_ITEM`	Client → Device	`bacnet_json_item_read()`	Return 200-byte chunk
`WRITE_SETTING_COMMAND`	Client → Device	(existing handler)	Set `webview_json_flash = 2`

Deploy sequence (per screen):

1. `WRITE_SETTING_COMMAND` → set `webview_json_flash = 2`
2. `WRITE_JSON_SCREEN` → device allocates buffer, stores `{ name, total_chunks, zip_size }`
3. `WRITE_JSON_ITEM` × N → device appends 200-byte chunks to buffer
4. Device verifies CRC, decompresses with zlib, parses JSON with cJSON
5. Device calls widget factory to create/destroy LVGL objects
6. Device calls `lv_scr_load()` if this is the active screen

Read sequence:

1. `READ_JSON_SCREEN` → device returns screen names + metadata from NVS
2. `READ_JSON_ITEM` × N → device reads 200-byte chunks from NVS, returns them

Files to create: `main/bacnet_json.c/h`, `main/screen_store.c/h`

3.1.2 REST API (HTTP on ESP32)

The ESP32 runs a lightweight HTTP server on port 8000 using ESP-IDF's `esp_http_server`:

```
TEXT

ESP32 Boot → rest_api_init()
  ├─ GET    /api/v1/screens                    → load all screens
  ├─ PUT    /api/v1/screens                    → deploy all screens
  ├─ GET    /api/v1/screens/:name              → load single screen
  ├─ PUT    /api/v1/screens/:name              → deploy single screen
  ├─ PATCH  /api/v1/screens/:name              → delta update
  ├─ PATCH  /api/v1/screens/:name/widgets/:id  → widget-level delta
  ├─ POST   /api/v1/images                     → upload bitmap image
  ├─ GET    /api/v1/images/:name               → download bitmap image
  └─ DELETE /api/v1/images/:name               → remove bitmap image
```

Handler details:

Handler	Action
`rest_get_screens()`	Read all screens from NVS, return `{ screens: [...], meta: {...} }`
`rest_put_screens()`	Parse body with cJSON, iterate screens, store each in NVS, return `{ deployed, failed, status }`
`rest_get_screen()`	Read single screen by name from NVS, return `{ name, json }`
`rest_put_screen()`	Store single screen to NVS, return `{ name, status }`
`rest_patch_screen()`	Load screen from NVS, apply dot-path changes (`widgets.btn.text`), save back
`rest_patch_widget()`	Load screen, navigate to `widgets.<id>.<path>`, apply change
`rest_push_image()`	Receive `{ name, data_base64 }`, decode base64, convert to LVGL format with `lv_img_conv`, store in NVS
`rest_pull_image()`	Read image from NVS, encode as base64, return
`rest_delete_image()`	Remove image from NVS

Key ESP-IDF APIs used:

- `httpd_start()` / `httpd_register_uri_handler()` — HTTP server
- `nvs_open()` / `nvs_set_str()` / `nvs_get_str()` / `nvs_commit()` — NVS storage
- `cJSON_Parse()` / `cJSON_Print()` / `cJSON_GetObjectItem()` — JSON parsing

Files to create: `main/rest_api.c/h`, `components/cJSON/cJSON.c/h`

3.1.3 NVS Storage Layout

```
NVS Namespace: "screens"
  Key: "index"      → JSON array of screen names ["Home", "Settings", ...]
  Key: "Home"       → full firmware JSON string for this screen
  Key: "Settings"   → full firmware JSON string for this screen
  ...

NVS Namespace: "images"
  Key: "bg_home"    → base64-encoded image data
  Key: "btn_heat"   → base64-encoded image data
  ...

NVS Namespace: "storage"
  Key: "WEBWIEW_JASON" → uint8 flag (0 = legacy, 2 = EEZ Studio)
```

Maximum NVS string size: ~4000 bytes per entry (ESP32 NVS limit). For larger screens, the BACnet path uses chunked transfer with zlib compression.

Flash budget (ESP32 32MB, partition `storage` at 0x760000):

Item	Size
`storage` partition	128 KB (0x20000)
Existing NVS usage (settings, WiFi, SNTP, etc.)	~5 KB
Free for screen JSON	~120 KB
Per-screen JSON (uncompressed)	~30 KB
Per-screen JSON (zlib compressed, typical)	5–10 KB
11 screens compressed	55–110 KB
Fits?	✅ Yes (tight for all 11; ~10 KB overhead remaining)

For projects larger than ~120 KB compressed, chunk the screens across multiple BACnet deploys or increase the `storage` partition size.

NVS entry size limit: ESP32 NVS has a ~4000-byte-per-entry limit. Screens larger than 4 KB compressed must be split across multiple NVS keys (e.g., `"Home_0"`, `"Home_1"`, ...) and reassembled on read. The BACnet chunked path (WRITE_JSON_ITEM) already handles this — the REST API should follow the same pattern for large screens.

Storage recommendations:

Concern	Suggestion
Including images (36 bitmaps × 3KB = ~108KB)	Bump `storage` partition to 384KB (`0x60000`). ESP32 has 32MB flash — plenty of room.
NVS 4KB-per-entry limit on images	Use SPIFFS or LittleFS partition for images instead of NVS. No per-entry size limit, better for binary blobs.
Partition layout (proposed)	Add a `spiffs` partition for images at ~0x7C0000, 256KB. Keep JSON screens in NVS `storage` .

3.1.4 Device Lifecycle Flow

First boot (factory state):

```
TEXT

Device powers on
  ├─ NVS "WEBWIEW_JASON" = 0 (legacy mode, or not set)
  ├─ No JSON in NVS namespace "screens"
  ├─ Load legacy SquareLine screens (compiled C code)
  └─ Display legacy UI
```

After first deploy (via BACnet or REST):

```
TEXT

Browser → Deploy screens
  ├─ WRITE_SETTING_COMMAND → webview_json_flash = 2
  ├─ WRITE_JSON_SCREEN + WRITE_JSON_ITEM × N → compressed JSON
  ├─ Device decompresses, parses JSON
  ├─ Device stores JSON in NVS "screens" namespace (survives reboot)
  ├─ Device sets webview_json_flash = 2 in NVS
  ├─ screen_manager_load_all() → creates LVGL screens from JSON
  └─ Display EEZ Studio UI
```

Normal boot (JSON already stored):

```
TEXT
```

```
Device powers on
  ├─ NVS "WEBWIEW_JASON" = 2
  ├─ Read screen index from NVS "screens"/"index"
  ├─ For each screen:
  │    ├─ Read JSON string from NVS
  │    ├─ Parse with cJSON
  │    ├─ Create LVGL screen (lv_obj_create)
  │    └─ widget_factory_create() → populate with widgets
  ├─ Load first screen: lv_scr_load(g_screens[0].screen)
  └─ Display EEZ Studio UI
```

Screen update (deploy after boot):

```
TEXT
```

```
Browser → Deploy updated screens
  ├─ Device receives JSON via BACnet/REST
  ├─ For each updated screen:
  │    ├─ Destroy existing LVGL objects for that screen
  │    ├─ Parse new JSON
  │    ├─ Store new JSON in NVS (overwrite)
  │    └─ widget_factory_create() → rebuild LVGL objects
  ├─ If updated screen is the active screen:
  │    └─ lv_scr_load() to refresh display
  └─ Response: { deployed: N, failed: 0, status: "ok" }
```

Fallback to legacy (if JSON is corrupted or missing):

```
TEXT
```

```
Device powers on
  ├─ NVS "WEBWIEW_JASON" = 2
  ├─ Failed to read/parse JSON from NVS
  ├─ Set webview_json_flash = 0 (revert to legacy)
  ├─ Load legacy SquareLine screens
  └─ Display legacy UI
```

Key design decisions:

Decision	Rationale
JSON stored in NVS, not RAM	Survives power loss and reboots
`webview_json_flash` flag	Fast check at boot: skip JSON loading if in legacy mode
Widgets recreated on every boot	No need to serialize LVGL object state — JSON is the source of truth
Fallback to legacy on parse error	Device always shows something, even if JSON is corrupt
Screen update destroys + recreates	Simplifies code — no incremental widget state tracking needed

3.2 Dynamic LVGL Rendering from Firmware JSON

The device parses firmware JSON at runtime and creates LVGL widgets dynamically — no recompilation needed.

3.2.1 Widget Factory

Mapping from firmware `sub_type` to LVGL create functions:

C

```c
// main/widget_factory.c
void widget_factory_create(lv_obj_t *parent, cJSON *widget_json) {
    const char *sub_type = cJSON_GetObjectItem(widget_json, "sub_type")->valuestring;
    lv_obj_t *obj = NULL;

    if      (strcmp(sub_type, "label") == 0)    { obj = lv_label_create(parent); }
    else if (strcmp(sub_type, "button") == 0)   { obj = lv_btn_create(parent); }
    else if (strcmp(sub_type, "arc") == 0)      { obj = lv_arc_create(parent); }
    else if (strcmp(sub_type, "bar") == 0)      { obj = lv_bar_create(parent); }
    else if (strcmp(sub_type, "image") == 0)    { obj = lv_image_create(parent); }
    else if (strcmp(sub_type, "switch") == 0)   { obj = lv_switch_create(parent); }
    else if (strcmp(sub_type, "slider") == 0)   { obj = lv_slider_create(parent); }
    else if (strcmp(sub_type, "dropdown") == 0) { obj = lv_dropdown_create(parent); }
    else if (strcmp(sub_type, "panel") == 0)    { obj = lv_obj_create(parent); }

    if (obj) {
        widget_factory_apply_position(obj, widget_json);    // x_pos, y_pos, width, heig
        widget_factory_apply_style(obj, widget_json);       // style definition
        widget_factory_apply_events(obj, widget_json);      // CLICKED → LV_EVENT_CLICKE
        widget_factory_apply_state_flags(obj, widget_json); // hidden, disabled
        widget_factory_apply_properties(obj, widget_json);  // type-specific props
        widget_factory_apply_children(obj, widget_json);    // recursive
    }
}
```

Sub-type properties:

sub_type	LVGL create	Properties applied
label	lv_label_create()	obj_text, long_mode, recolor
button	lv_btn_create()	obj_text (inner label), inner_align, checked
arc	lv_arc_create()	min, max, value, mode
bar	lv_bar_create()	min, max, value, mode
image	lv_image_create()	src, rotation, pivot_x/y
switch	lv_switch_create()	checked
slider	lv_slider_create()	min, max, value, mode
dropdown	lv_dropdown_create()	options[], selected
panel	lv_obj_create()	children (container only)

3.2.2 Expression Engine

Resolves dynamic expressions to Modbus register values:

```c
// main/expression.c
float expression_eval(const char *expr) {
    if (strncmp(expr, "variables.", 10) == 0)
        return modbus_read_variable(expr + 10);
    if (strncmp(expr, "zones[", 6) == 0)
        return modbus_read_zone(expr);
    return (float)atof(expr);
}
```

Pattern	Example	Resolution
variables.<name>	variables.temp	Modbus register lookup
zones[N].<field>	zones[0].setpoint	Array + field access

3.2.3 Screen Manager

```c
// main/screen_manager.c
typedef struct { char name[32]; lv_obj_t *screen; cJSON *json; } device_screen_t;
device_screen_t g_screens[MAX_SCREENS];
int g_screen_count = 0;

void screen_manager_load_all(void) {
    // Read screen index from NVS, load each screen JSON, create LVGL objects
}
void screen_manager_switch(const char *name) {
    // lv_scr_load_anim(g_screens[i].screen, LV_SCR_LOAD_ANIM_FADE_ON, 200, 0, false);
}
void screen_manager_reload(const char *name) {
    // After BACnet deploy: destroy old screen, re-parse JSON, recreate widgets
}
```

3.2.4 Action Engine

```c
// main/action_engine.c
void action_engine_execute(cJSON *actions) {
    cJSON *action; cJSON_ArrayForEach(action, actions) {
        const char *type = cJSON_GetObjectItem(action, "action")->valuestring;
        if (strcmp(type, "changeScreen") == 0)
            screen_manager_switch(cJSON_GetObjectItem(action, "screen")->valuestring);
        else if (strcmp(type, "setVariable") == 0)
            modbus_write_variable(cJSON_GetObjectItem(action, "variable")->valuestring,
                                  cJSON_GetObjectItem(action, "value")->valuedouble);
    }
}
```

Files to create: `main/widget_factory.c/h`, `main/expression.c/h`, `main/screen_manager.c/h`, `main/action_engine.c/h`

3.3 Export Current Device Interface to JSON

The current firmware has 11 SquareLine Studio screens compiled as C code. They need to be exported to firmware JSON format so the frontend can load and render them.

3.3.1 Export Approach

SquareLine screens are C code — recreate each screen in EEZ Studio manually, then deploy:

```
TEXT
SquareLine .sls project → .c/.h files (in firmware)
       ↓ manually recreate in EEZ Studio
EEZ Studio .eez-project
       ↓ Deploy to Device (firmware-export.ts → firmware JSON)
firmware JSON per screen → NVS storage → Frontend can Import
```

3.3.2 Screen Mapping

SquareLine Screen	Firmware JSON	Widgets
`ui_HomeScreen`	`home`	Temperature, humidity, fan/mode buttons, WiFi icon
`ui_MainMenu`	`main_menu`	Navigation buttons to sub-screens
`ui_WifiConfig`	`wifi_config`	SSID, password, scan, IP fields
`ui_NetworkConfig`	`network_config`	IP, subnet, BACnet port
`ui_Protocols`	`protocols`	Protocol toggles
`ui_Parameters`	`parameters`	Read-only labels
`ui_Time`	`time`	Clock, timezone
`ui_ScheduleScreen`	`schedule`	Program list
`ui_ScheduleEditScreen`	`schedule_edit`	Day/time/temp editor
`ui_HolidayCalenderScreen`	`holidays`	Holiday picker
`ui_StartUpScreen`	`startup`	Logo

3.3.3 Widget Conversion Example

```c
// SquareLine C code:
ui_TemperatureVal = lv_label_create(ui_HomeScreen);
lv_label_set_text(ui_TemperatureVal, "22.5");
lv_obj_set_pos(ui_TemperatureVal, 80, 20);
lv_obj_set_style_text_font(ui_TemperatureVal, &ui_font_Arial80, LV_PART_MAIN);
```

JSON

```
{
  "sub_type": "label", "x_pos": 80, "y_pos": 20,
  "obj_text": "22.5", "text_type": "literal",
  "style": { "DEFAULT": { "text_font": "Arial80" } }
}
```

3.3.4 Image Assets

SquareLine embeds images as C arrays (`ui_img_fan_auto_new_png.c`). The `lv_img_conv_v9` tool converts them to LVGL format. Image data is sent separately via `/api/v1/images` (REST) or `/api/eez-device/images/push/:panelId` (BACnet) and referenced by name in screen JSON. Bitmap names in the firmware JSON match the SquareLine asset names.